

Azure Cloud Infrastructure

Presentation & Lab Guide

VM Connectivity · Web Hosting · Free Account Machines · Auto-Shutdown

Simon Max – Würth Eric

Contents

1 — VM Connectivity	4
1.0 Connection Methods — Quick Reference	4
1.1 Initial Setup	4
1.2 Connect via Bastion	4
1.3 Connect via External SSH	4
1.4 Connect via Azure Cloud Shell (with Public IP)	5
1.5 Remove the Public IP Address	5
Dissociate the Public IP	5
Delete the Public IP	5
1.6 Verify Bastion Still Works (After IP Removal)	5
1.7 Attempt SSH via Cloud Shell — Without VNet Integration	5
1.8 Attempt SSH via Cloud Shell — With Existing VNet (First Try)	6
1.9 Why a Relay and Network Profile Are Required	6
Data Flow — Before and After Setup	6
Components Explained	6
1.10 Create the Network Profile and Relay Namespace	7
Step A — Register Resource Providers	7
Step B — Get the Azure Container Instance Object ID	7
Step C — Deploy Cloud Shell VNet Template	7
Step D — Deploy Cloud Shell VNet Storage Template	8
1.11 Connect via Cloud Shell with VNet Integration (Success)	9
1.12 Connect via a Jump Host (Second VM)	9
2 — Web Hosting with a VM	10
2.1 Setup	10
2.2 Remove the Public IP from VM1	10
Dissociate	10
Delete	10
2.3 Install nginx as a Reverse Proxy on VM2	10
2.4 Install nginx as a Web Server on VM1	11
2.5 Access the Website	12
3 — Auto-Shutdown	13
4 — Free Account Virtual Machines	14
4.1 Creation Steps	14
4.2 Free Tier Restrictions	14

5 — Azure Resource Visualizer	15
5.1 What it does	15
5.2 Reading the Diagram	15
5.3 What is Shown in This Diagram	16
Left Cluster — VM1 (Private, No Public IP)	16
Centre — Virtual Network and Cloud Shell Infrastructure	16
Right Cluster — VM2 (Public, Jump Host / Reverse Proxy)	17
6 — Lucidchart Architecture Diagram	18
6.1 Diagram	18
6.2 What the Diagram Shows	18
6.3 Link	19

1 — VM Connectivity

1.0 Connection Methods — Quick Reference

Use this table to understand at a glance which method to use and when. The lab will walk through each one in order.

Method	Needs Public IP	Needs Bastion	Works w/o Public IP	Best used when
Azure Bastion	No	Yes	Yes	Quick GUI access; no SSH client needed
External SSH	Yes	No	No	Normal SSH from your local machine
Cloud Shell (public IP)	Yes	No	No	Quick terminal in the browser
Cloud Shell + VNet	No	No	Yes	Full private access via Relay tunnel
Jump Host (VM2)	No	No	Yes	Alternative when VNet setup is unavailable

* Jump Host VM2 does need its own public IP, but VM1 (your target) does not.

1.1 Initial Setup

- Create a VM and a VNet inside the "Create a resource" menu (steps are already known to the class).

1.2 Connect via Bastion

How it works: Bastion is a managed service that runs inside Azure's own network. It provides a browser-based SSH/RDP session without ever exposing your VM to the public internet. Traffic flows over the Azure backbone — your VM does not need a public IP.

- Navigate to your Resource Group.
- Click on your VM → click the Connect button → choose Connect via Bastion.
- Enter your username and password, then click Connect.

1.3 Connect via External SSH

How it works: Classic SSH — your local machine opens a TCP connection directly to the VM's public IP on port 22. This requires the VM to have a public IP address and port 22 open in the Network Security Group.

- Open any remote access program that supports SSH.
- Open an SSH session, enter your username and password, and click Connect.

1.4 Connect via Azure Cloud Shell (with Public IP)

How it works: Azure Cloud Shell is a browser-based terminal hosted by Microsoft. By default it runs in Microsoft's managed network — separate from your VNet — so it can only reach your VM through the public internet (i.e., via the VM's public IP).

- Open Cloud Shell from the Azure Portal.
- When prompted, choose Bash.
- Select your subscription and leave the rest as default → click Apply.
- Type the following command:

```
ssh {username}@{yourPublicIP}
```

1.5 Remove the Public IP Address

Dissociate the Public IP

- Go to your Resource Group → click on your VNet resource → click Dissociate → confirm Yes.

Delete the Public IP

- Go to your Resource Group → click on your VNet resource → click Delete → confirm Yes.

1.6 Verify Bastion Still Works (After IP Removal)

This step demonstrates one of Bastion's key advantages: because it operates at the Azure network level, removing the public IP has no effect on it. Students should confirm this before attempting the Cloud Shell steps.

- Go to Resource Group → click your VM → Connect → Connect via Bastion.
- Enter credentials and connect. Bastion works independently of a public IP.

1.7 Attempt SSH via Cloud Shell — Without VNet Integration

Expected result: this will fail. Cloud Shell is in Microsoft's managed network and has no route to your VM's private IP. This step exists to show why the next steps are needed.

- Open Cloud Shell → choose Bash.
- Select subscription → leave defaults → click Apply.

- Attempt the connection:

```
ssh {username}@{yourPrivateIP}    # This will time out — no route to host
```

1.8 Attempt SSH via Cloud Shell — With Existing VNet (First Try)

- Open Cloud Shell → click Settings → choose Bash.
- Select your subscription.
- Check "Use an existing private virtual network" → click Apply.
- Select your Resource Group and VNet.
- Notice the error: no Network Profile and no Relay Namespace are present.
- Click Previous → close Cloud Shell.

Cloud Shell cannot attach to your VNet without a Relay Namespace and a Network Profile. The next section explains exactly what these are and why they are needed.

1.9 Why a Relay and Network Profile Are Required

When Cloud Shell is configured to use an existing VNet, it runs as an Azure Container Instance (ACI) that is injected directly into one of your subnets. This gives the shell a private IP address on your network — but it introduces two infrastructure dependencies that must exist first.

Data Flow — Before and After Setup

Without Relay (fails)	With Relay (succeeds)
Cloud Shell (Microsoft network) ↓ SSH to private IP ↓ ??? (no route) ↓ VM1 (private IP)	Cloud Shell container ↓ (attached to subnet via ACI) Relay Namespace (HTTPS tunnel) ↓ (outbound, no inbound ports needed) Your VNet subnet (private) ↓ VM1 (private IP)

Key insight: the Relay Namespace replaces the public IP as the entry point. Instead of opening an inbound port, it creates an outbound HTTPS/WebSocket tunnel (Azure Relay Hybrid Connection) from inside the VNet back to Cloud Shell. No firewall rules need to be opened.

Components Explained

Component	What it does & why it is needed
Azure Relay Namespace	Creates an encrypted outbound tunnel (Hybrid Connection over HTTPS/WebSocket) between the Cloud Shell container and your VNet. Because the tunnel is outbound-only, no inbound firewall rules need to be opened on your VNet. This is what replaces the public IP as an entry point.
Network Profile	Defines which subnet the Cloud Shell container will attach to inside your VNet. It acts as the network configuration template for the Azure Container Instance — specifying subnet, IP allocation, and DNS settings so the ACI receives a private IP on your network.
Microsoft.ContainerInstance provider	The Azure Container Instance resource provider must be registered on the subscription before any ACI resource can be deployed. Without registration, Azure will reject the deployment entirely.
Microsoft.Relay provider	The Azure Relay resource provider must similarly be registered before a Relay Namespace can be created. This is a one-time step per subscription and is easy to forget.

The VNet and the Resource Group used for the Cloud Shell templates must be in the same Azure region as your VM. Mismatched regions are the most common reason this deployment fails.

1.10 Create the Network Profile and Relay Namespace

Before deploying, the required resource providers must be registered on the subscription, and the Azure Container Instance Object ID must be retrieved.

Step A — Register Resource Providers

- Go to your Subscription → select your subscription.
- In the left nav bar click Resource providers.
- Search for Microsoft.ContainerInstance → click the three-dot menu → Register.
- Repeat for Microsoft.Relay.

Step B — Get the Azure Container Instance Object ID

- In the search bar find Microsoft Entra ID → click it.
- Find the Object ID field and copy the value.

Step C — Deploy Cloud Shell VNet Template

- Navigate to: <https://learn.microsoft.com/de-de/samples/browse/>
- Search for cloud-shell-vnet → click the result that does NOT say "storage".
- Click Deploy to Azure and fill in the following fields:

Field	Value
Subscription	Yours
Resource group	Your Norway East group (with your VM)
Existing VNET Name	Your VNet's name
Relay Namespace Name	e.g. cloudshell-relay-01
NSG Name	e.g. cloudshell-nsg
Azure Container Instance OID	Paste from Step B
Container Subnet Name	cloudshellsubnet
Container Subnet Address Prefix	10.0.1.0/24
Relay Subnet Name	relaysubnet
Relay Subnet Address Prefix	10.0.2.0/24
Storage Subnet Name	storagesubnet
Storage Subnet Address Prefix	10.0.3.0/24
Private Endpoint Name / Tags / Location	Leave as default

- Click Review + Create.

Step D — Deploy Cloud Shell VNet Storage Template

- Return to <https://learn.microsoft.com/de-de/samples/browse/>
- Search for cloud-shell-vnet-storage → click the result → Deploy to Azure.
- Fill in the following fields:

Field	Value
Subscription	Yours
Resource group	Same Norway East group
Existing VNET Name	Your VNet's name
Existing Storage Subnet Name	storagesubnet
Existing Container Subnet Name	cloudshellsubnet
Storage Account Name	Globally unique, all lowercase, no special characters
File Share Name	Leave default: acsshare

Tags / Location

Leave as default

- Click Review + Create.

1.11 Connect via Cloud Shell with VNet Integration (Success)

After completing the setup above, Cloud Shell will start inside your private subnet and will have a direct route to VM1's private IP — no public IP required.

- Open Cloud Shell → click Settings → Reset user settings.
- Choose Bash → select your subscription.
- Check "Use an existing private virtual network" → click Apply.
- Select your VNet, the appropriate subnets, and the storage account → click Select.
- Once the shell initialises, SSH to your VM:

```
ssh {username}@{yourPrivateIP}
```

1.12 Connect via a Jump Host (Second VM)

How it works: VM2 acts as a stepping stone. You SSH into VM2 (which has a public IP), and from there SSH into VM1 (which only has a private IP). VM2 can reach VM1 because they share the same VNet and therefore the same private address space.

- Create a second VM (VM2) with a public IP in the same region.
- SSH into VM2 from Cloud Shell:

```
ssh {username}@{PublicIP-VM2}
```

- From inside VM2's shell, SSH to VM1 using its private IP:

```
ssh {username}@{PrivateIP-VM1}
```

1.13 Connect to the VM using an SSH key

Step A — Using an SSH key on a new virtual machine

- Click on create a resource.
- Click on create under Virtual machine
- Do everything like you did with the VM's we created before, but at Authentication type choose SSH public key.

- After creating the VM you will be asked to download the SSH key, click yes.
- Open your Cloud Shell and type in the command "ssh -i <private-key-file-path> {yourUser}@{yourPublicIP}".

Step B — Changing from password to an SSH key on an already existing VM.

- On your personal Computer create an Public and Private key using the command "ssh-keygen -t rsa -b 4096 -C "your@email.com" -f ~/.ssh/azure_vm_key" (never share your private key)
- You should now have a private key under "~/.ssh/azure_vm_key" and a public key under "~/.ssh/azure_vm_key.pub".
- Now connect to your VM using ssh or Bastion and save your public key using the commands "echo "<Your_PUBLIC_KEY>" >> /home/<username>/.ssh/authorized_keys", "chmod 600 /home/<username>/.ssh/authorized_keys", "chmod 700 /home/<username>/.ssh".
- Now you can connect to your VM using the ssh key using the command "ssh -i ~/.ssh/azure_vm_key <username>@<vm-ip-oder-dns>"

2 — Web Hosting with a VM

DMZ concept: VM1 (the web server) is intentionally isolated from the public internet — it has no public IP. VM2 (the reverse proxy) is the only publicly reachable entry point. It forwards legitimate HTTP requests inward to VM1. This means attackers cannot reach VM1 directly even if they discover its private IP, because there is no route from the internet to that address.

2.1 Setup

- Create 2 VMs with VNets via "Create a resource" (ensure port 80 is open on both). Students already know the process.

2.2 Remove the Public IP from VM1

Dissociate

- Go to Resource Group → click your VNet resource → click Dissociate → confirm Yes.

Delete

- Go to Resource Group → click your VNet resource → click Delete → confirm Yes.

2.3 Install nginx as a Reverse Proxy on VM2

nginx on VM2 listens on port 80 and forwards all requests to VM1's private IP. From the outside world, only VM2 is visible.

- Connect to VM2 using Bastion.
- Install nginx:

```
sudo apt install -y nginx
```

- Edit `/etc/nginx/sites-available/default` and replace the contents with:

```
server {  
    listen 80;  
    location / {  
        proxy_pass http://10.0.0.4;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    }  
}
```

- Before restarting nginx, test the config for syntax errors:

```
sudo nginx -t
```

Always run `nginx -t` after editing the config. If there is a typo, nginx will show the exact line number.

- Apply the configuration:

```
sudo systemctl restart nginx
```

2.4 Install nginx as a Web Server on VM1

- Connect to VM1 using Bastion.
- Install nginx:

```
sudo apt install -y nginx
```

- Edit `/var/www/html/index.html` and add your HTML content.
- Test and restart nginx the same way:

```
sudo nginx -t  
sudo systemctl restart nginx
```

2.5 Access the Website

- Open a browser and navigate to VM2's public IP address.
- nginx on VM2 forwards the request through the private network to VM1, and the page is served.

3 — Auto-Shutdown

Auto-shutdown is a cost-saving feature that powers off a VM at a scheduled time each day. It does not deallocate resources automatically on a timer — you set it once and it runs daily.

- Go to Virtual machines → choose your VM.
- Under Operations, select Auto-shutdown.
- Set Enabled to On.
- Set the desired shutdown time.
- Select the timezone: (UTC +01:00) Belgrade.
- Optionally enable notifications (via Webhook URL or Email address).
- Click Save.

4 — Free Account Virtual Machines

4.1 Creation Steps

- In the search bar, search for free account virtual machine.
- Click the third result (the first shows only Windows ISOs, the second only Linux, the third shows both).
- Select your Subscription → click Create.
- Create a new Resource Group. Existing resources are not allowed, and only one free VM is permitted per Resource Group.
- Give the Resource Group a name → click OK.
- Set the Virtual Machine name.
- Select a Region.
- Select the OS Image.
- Select one of the two available free-tier VM sizes.
- Choose Password as the Administrator account type.
- Set a Username and Password.
- Under Inbound ports, select Allow selected ports → choose SSH (22).
- Click Next: Tags > — optionally add tags → click Review + Create.
- Wait for validation. If it fails, the most common causes are an unsupported machine size or an ineligible region.
- When validated, click Create.

This VM is usable for 750 hours or 12 months from account registration, whichever comes first. The 750 hours are shared between all VM's in the same subscription. Every VM needs its own resource group.

4.2 Free Tier Restrictions

Category	Details
Operating Systems	Windows Server 2022 · Debian 11 · AlmaLinux 9 · Ubuntu Server 22.04 LTS · Ubuntu Server 24.04 LTS
VM Sizes	B1s (1 vCPU, 1 GB RAM) · B2ats_v2 (2 vCPU, 1 GB RAM)
Usage Hours	750 hours total across all VMs, or 12 months — whichever comes first
Storage	2 × 64 GB SSD
Networking	15 GB outbound data free/month · Inbound traffic free · Public IP addresses are NOT free
Common Regions	East US · West US · West Europe
Limitations	No auto-scaling · Load Balancer included · No SLA guaranteed · Free tier lasts 12 months from registration · Only 1 free subscription per Microsoft account

5 — Azure Resource Visualizer

The Azure Resource Visualizer is a built-in tool in the Azure Portal that generates an interactive map of all the resources inside a Resource Group and shows how they are connected to each other. It is accessed directly from the Resource Group overview page.

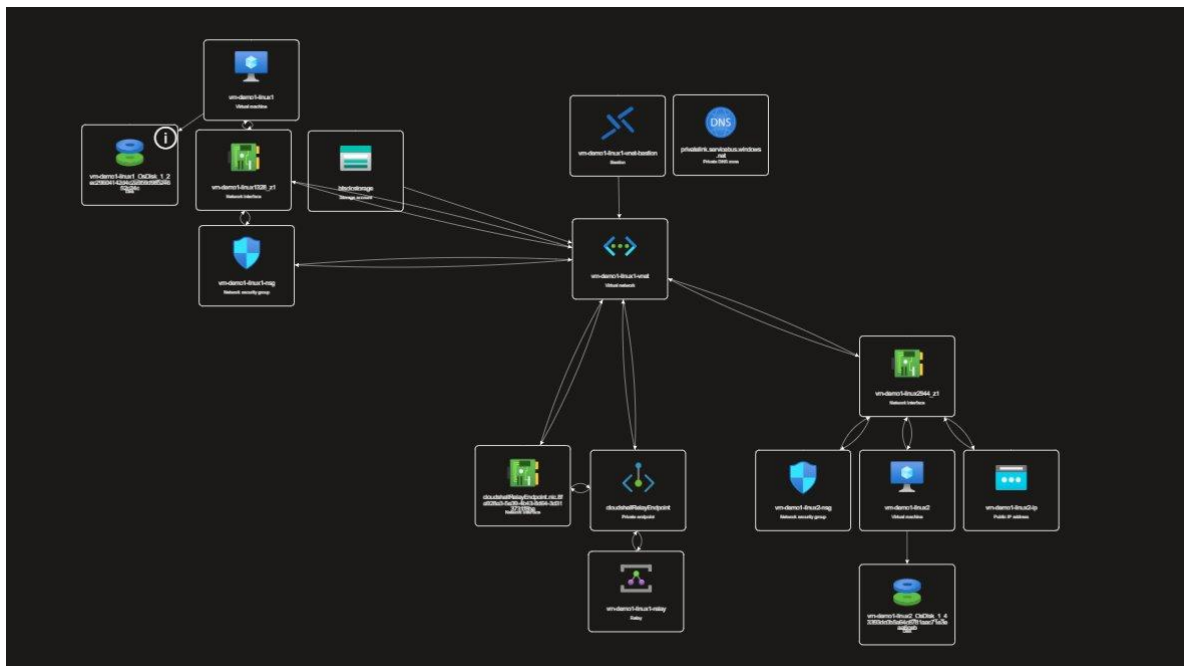
5.1 What it does

The Resource Visualizer reads the relationships between your Azure resources and draws them as a node graph — each resource appears as a labelled icon, and lines between them show dependencies and connections. This makes it immediately obvious which resources are linked, which belong to the same network, and what the overall shape of your deployment looks like without having to open each resource individually.

- Shows all resources in the Resource Group as visual nodes.
- Draws dependency lines between connected resources (e.g. a VM linked to its NIC, its NSG, its VNet, its storage account).
- Updates live — any resource you add or remove appears automatically.
- Lets you click any node to jump directly to that resource.
- Useful for quickly checking that all expected connections exist and spotting resources that are unexpectedly isolated.

5.2 Reading the Diagram

The screenshot below shows the Resource Visualizer for the demo environment used in this presentation. The central node is the Virtual Network (VNet), which acts as the hub connecting everything else.



5.3 What is Shown in This Diagram

The diagram contains two main clusters of resources, both connected through the central Virtual Network (vm-demo1-linux1-vnet).

Left Cluster — VM1 (Private, No Public IP)

This is the primary VM that was configured throughout the lab. It has no public IP address, which is why all the extra networking infrastructure was needed.

Resource	What it is
vm-demo1-linux1	The main Virtual Machine (VM1). No public IP — only reachable via Bastion, Cloud Shell VNet, or jump host.
vm-demo1-linux1328_z1	The Network Interface Card (NIC) attached to VM1. Holds its private IP address and connects the VM to the VNet.
vm-demo1-linux1-nsg	The Network Security Group (NSG) for VM1. Defines which traffic is allowed in and out (e.g. port 22 for SSH).
blobstorage	A Storage Account linked to the Cloud Shell VNet setup — used as the Cloud Shell file share (acsshare).
vm-demo1-linux1_OsDisk_1_2...	The OS disk attached to VM1. Shown as a dependency of the VM.

Centre — Virtual Network and Cloud Shell Infrastructure

The VNet is the hub of the entire diagram. The Cloud Shell relay infrastructure branches downward from it.

Resource	What it is
vm-demo1-linux1-vnet	The Virtual Network. Every other resource connects through this. It contains all the subnets: the VM subnet, the Cloud Shell container subnet, the relay subnet, and the storage subnet.
vm-demo1-linux1-eni-bastion	The Azure Bastion resource. Connected to the VNet so it can reach VM1 over the private network without needing a public IP on the VM.
privatelink.servicebus.windows.net	A Private DNS Zone used by the Relay Namespace. Ensures that the relay's internal hostname resolves to a private IP rather than a public one, keeping relay traffic inside the VNet.
cloudshellRelayEndpoint	The Private Endpoint for the Azure Relay. This is the network entry point through which Cloud Shell communicates with the Relay Namespace over the private network.

cloudshellRelayEndpoint.nic...	The NIC attached to the Private Endpoint — gives it a private IP address inside the VNet.
vm-demo1-linux1-relay	The Azure Relay Namespace itself. Creates the secure outbound tunnel (Hybrid Connection) that Cloud Shell uses to reach the private network. This is the resource explained in Section 1.9.

Right Cluster — VM2 (Public, Jump Host / Reverse Proxy)

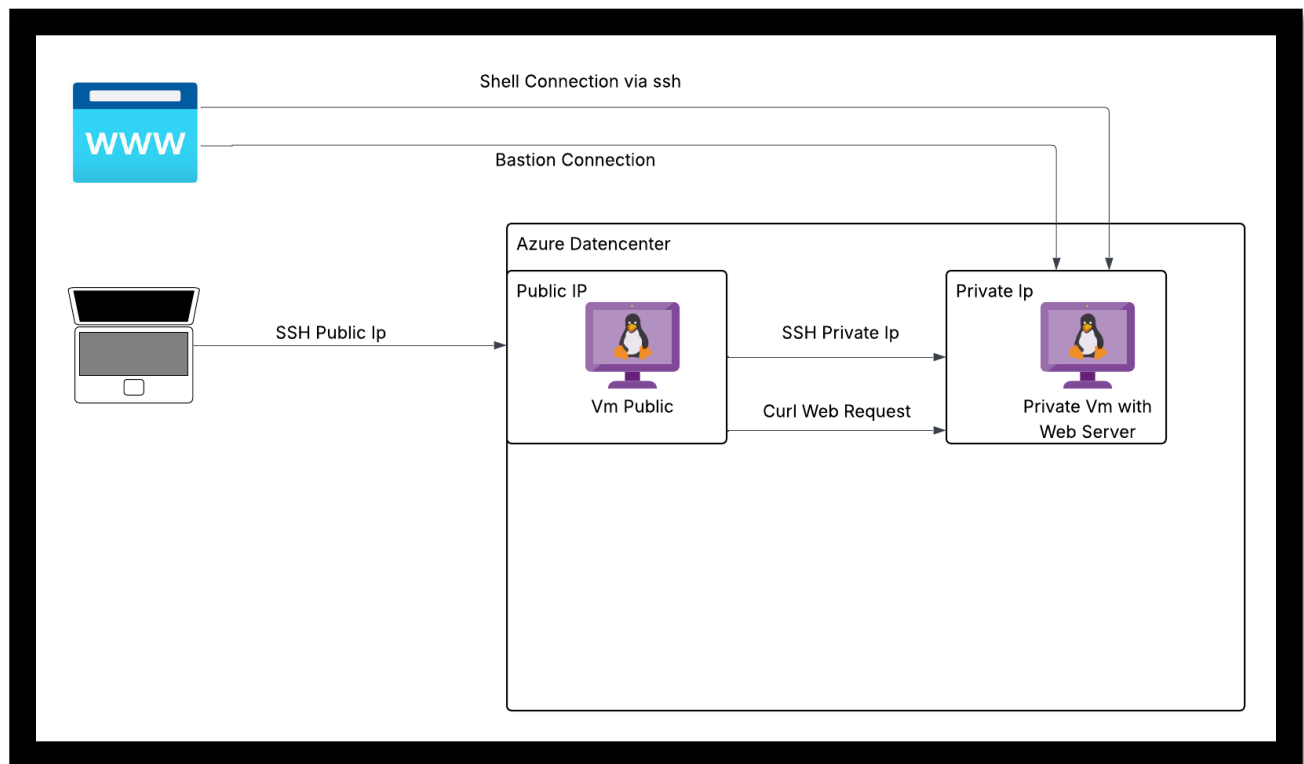
This is the second VM used in both the web hosting section (as the nginx reverse proxy) and the jump host method (as the SSH stepping stone).

Resource	What it is
vm-demo1-linux2	VM2 — the only VM with a public IP. Acts as the internet-facing entry point for both web traffic (nginx reverse proxy) and SSH access (jump host).
vm-demo1-linux2544_z1	The Network Interface Card (NIC) for VM2. Connected to the same VNet as VM1, which is what allows VM2 to reach VM1 via private IP.
vm-demo1-linux2-nsg	The Network Security Group for VM2. Port 80 (HTTP) and port 22 (SSH) must be open here for the web hosting and jump host labs to work.
vm-demo1-linux2-ip	The Public IP Address resource attached to VM2. This is what makes VM2 reachable from the internet.
vm-demo1-linux2_OsDisk_1_4...	The OS disk for VM2.

6 — Lucidchart Architecture Diagram

The Lucidchart diagram below provides a cleaner, hand-drawn view of the same architecture.

6.1 Diagram



6.2 What the Diagram Shows

The diagram maps out the same two-VM architecture from a network design perspective, showing how traffic flows between the internet, the VNet, and the individual VMs.

- The internet connects to VM2's public IP — this is the only entry point from outside.
- VM2 sits in the shared VNet subnet. It forwards HTTP traffic to VM1 (nginx reverse proxy) and can be used as an SSH jump host to reach VM1.
- VM1 is fully private — no public IP, reachable only through VM2 or via Bastion/Cloud Shell VNet.
- Azure Bastion connects to the VNet and provides direct browser-based access to both VMs without going through the public internet.
- The Relay Namespace and Cloud Shell infrastructure are attached to dedicated subnets within the VNet, allowing Cloud Shell to reach VM1 privately.

6.3 Link

[Lucid Chart link](#)